Proving Basic Algorithms Correct: Sorting

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Proving Basic Algorithms Correct: Sorting
Computer-Assisted Theorem Proving (7CCSACTP)
Mohammad Abdulaziz

Merge Sort: Top-Down

```
merge :: 'a list ⇒ 'a list ⇒ 'a list
```

```
msort :: 'a list ⇒ 'a list
```


Merge Sort: Top-Down
merge :: 'a list ⇒ 'a list ⇒ 'a list
msort :: 'a list ⇒ 'a list

- Top-down merge sort splits the list into halves, sorts each half recursively, and merges them.
- merge is defined over two sorted lists by comparing heads.
- msort divides using length xs div 2.
- Example: `msort [4,3,1,5]` splits into `take 2 = [4,3]` and `drop 2 = [1,5]`, sorts them to `[3,4]` and `[1,5]`, and merges them to `[1,3,4,5]`.
- **Definitions:**

```
merge [] ys = ys
merge xs [] = xs
merge (x # xs) (y # ys) =
  (if x ≤ y then x # merge xs (y # ys)
   else y # merge (x # xs) ys)
msort xs = (let n = length xs in
  if n ≤ 1 then xs
  else merge (msort (take (n div 2) xs))
             (msort (drop (n div 2) xs)))
```